

Gima Whitepaper

A local-first AI workspace for memory, research, artifact creation, media workflows, and safe self-improvement.



GIMA

Version	Date	Author	Project
2.2	4 July 2026	Gimhan Gunarathne	Gima / Human AI Local

Gima is designed as an inspectable AI workspace: local memory, visible outputs, optional cloud model routing, browsing for current facts, and review-gated improvement.

Repository: <https://github.com/data-G/Gima>

Contents

Section	Focus
1. Executive Summary	Product promise and principles
2. Product Vision	What Gima should become
3. Architecture	Routing, memory, models, and outputs
4. Product Screenshots	Current UI evidence and workflow examples
5. Capability Model	Browsing, artifacts, media, agents, and improvement
6. Safety and Roadmap	Governance, earning direction, and next milestones

Executive Summary

Gima is a local-first AI workspace built to make a personal AI useful, inspectable, and controllable on everyday hardware. It combines local memory, a browser-based chat interface, optional cloud model gateways, source-backed browsing, artifact generation, media planning, and review-gated self-improvement.

Principle	Meaning
Local-first memory	Conversations, knowledge, outputs, reviews, and logs live in the local Gima workspace.
Hybrid intelligence	Gima routes tasks to local models, browsing, deterministic artifact tools, or optional cloud providers.
Real artifacts	Reports, CSV files, Markdown, PDFs, JPG previews, workbooks, and media plans are written to visible folders.
Review-gated improvement	Gima can plan and test changes, but system upgrades require explicit user control.

The design goal is not to pretend that one small model can do everything. The goal is to route each request to the most reliable mechanism.

Problem and Product Vision

Many AI assistants are powerful but opaque. Users often cannot inspect where memory is stored, which provider answered, whether a generated file exists, or whether current facts were actually checked.

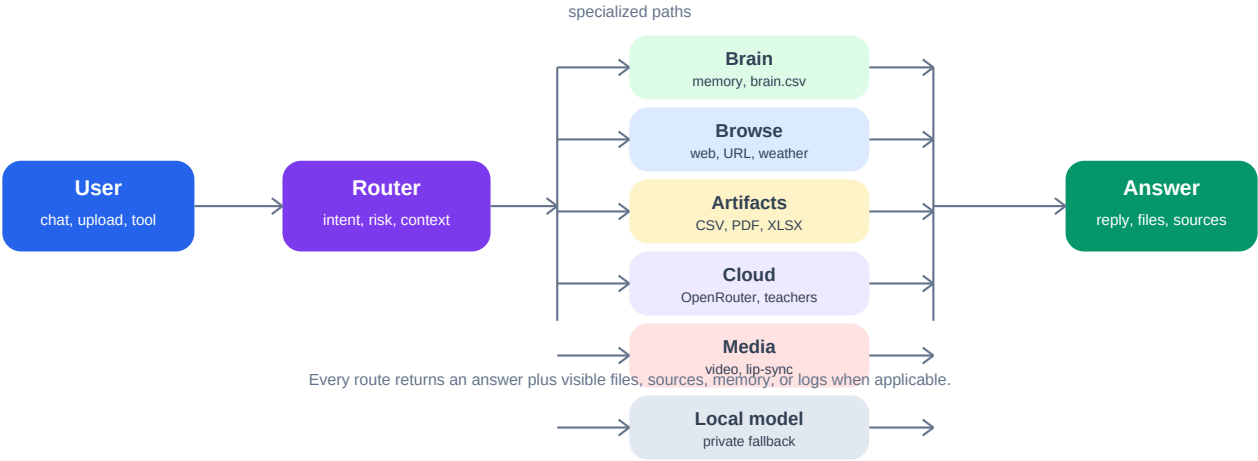
- Answer from local memory with traceable context.
- Browse the public web when current information is needed.
- Create business-grade reports, costing tables, PDFs, JPG previews, and workbooks.
- Use linked providers through OpenRouter, OpenAI, Gemini, Anthropic, and other gateways.
- Plan media workflows with scene beats, camera angles, emotion, audio timing, and lip-sync direction.

Workspace Area	Purpose
Brain	Long-term local knowledge and retrieval
Hands/in	Uploaded files that become part of the next prompt
Hands/out	Generated reports, media drafts, manifests, and exports
Continuous	Daily improvement logs, plans, and snapshots

Architecture and Routing

Gima is organized around a routing layer. A user request is classified and sent to the best available path: memory, browsing, artifact generation, media workflow, cloud model gateway, or local fallback.

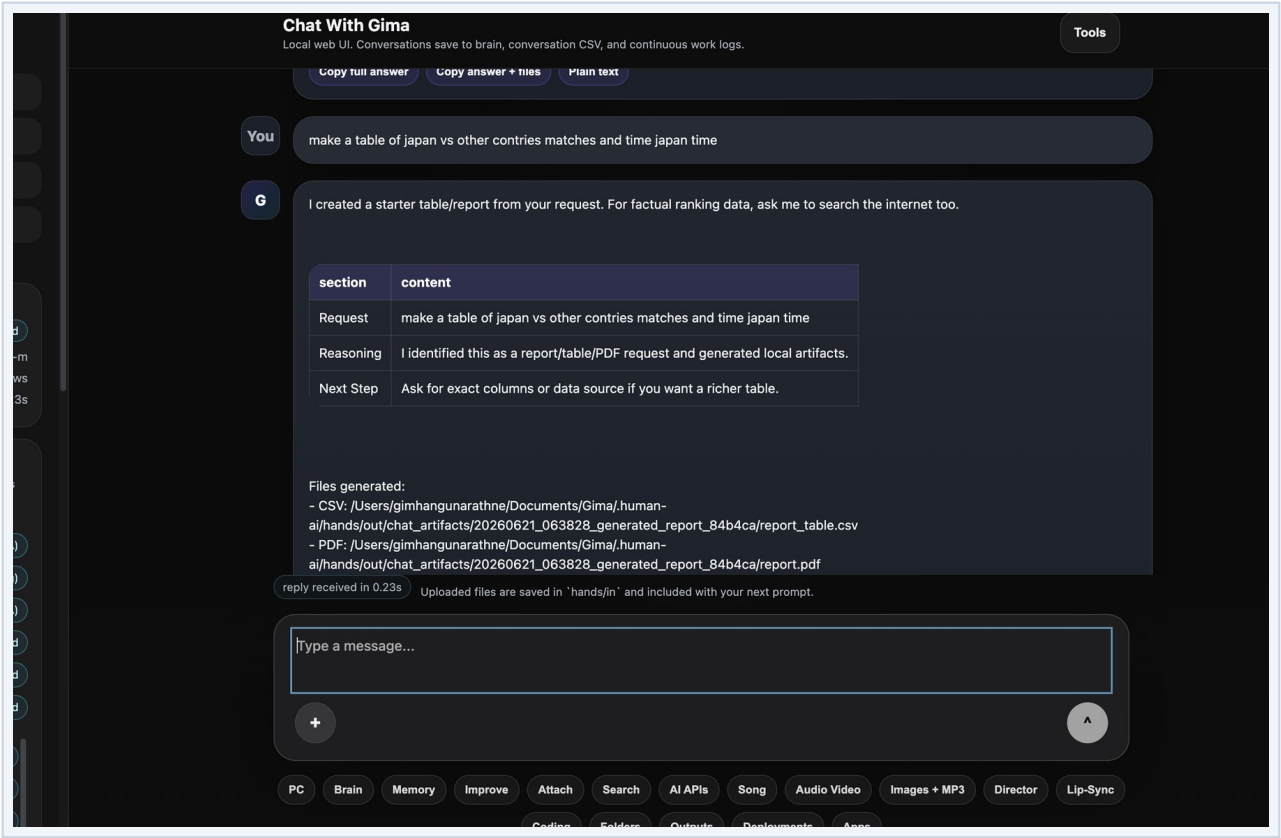
Figure 1 - Gima Routing Architecture



Layer	Responsibility	Implementation
Interface	Chat, uploads, dashboards, tools	web_ui.py, Start Gima.command
Memory	Records, reviews, conversations, brain index	.human-ai, memory.py, brain_index.py
Routing	Choose brain, browse, artifact, media, cloud, or local path	web_ui.py, agent.py, artifacts.py
Browsing	Public search, direct URL import, current weather	WebImporter, DuckDuckGo Lite, Open-Meteo
Cloud gateway	Optional stronger model access	OpenAI, Gemini, Anthropic, OpenRouter
Improvement	Backup, plan, patch, test, GitHub sync	self_update.py, vibe_code.py, scripts

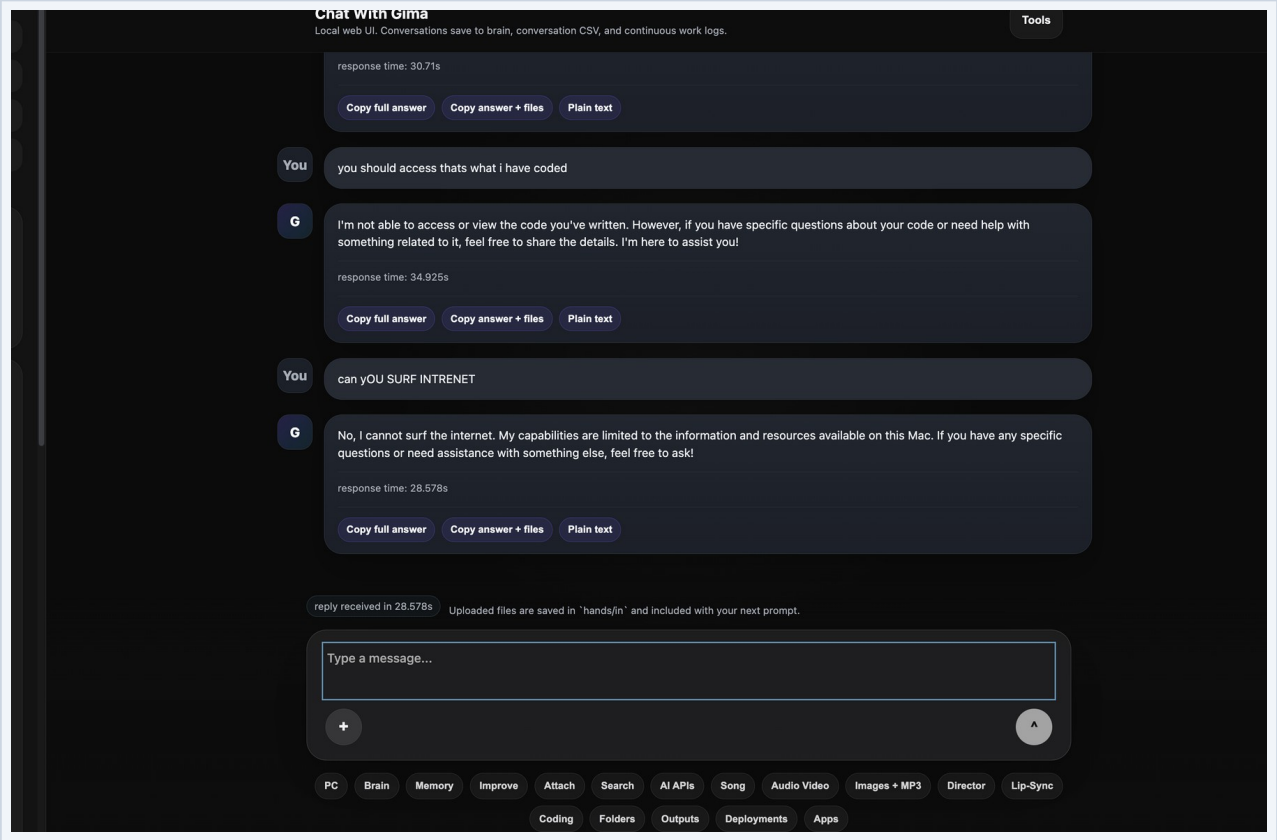
Product Screenshots

The screenshots show the current local web UI and the direction of the product experience: chat-first interaction, visible output generation, provider-aware controls, and media workflow buttons.



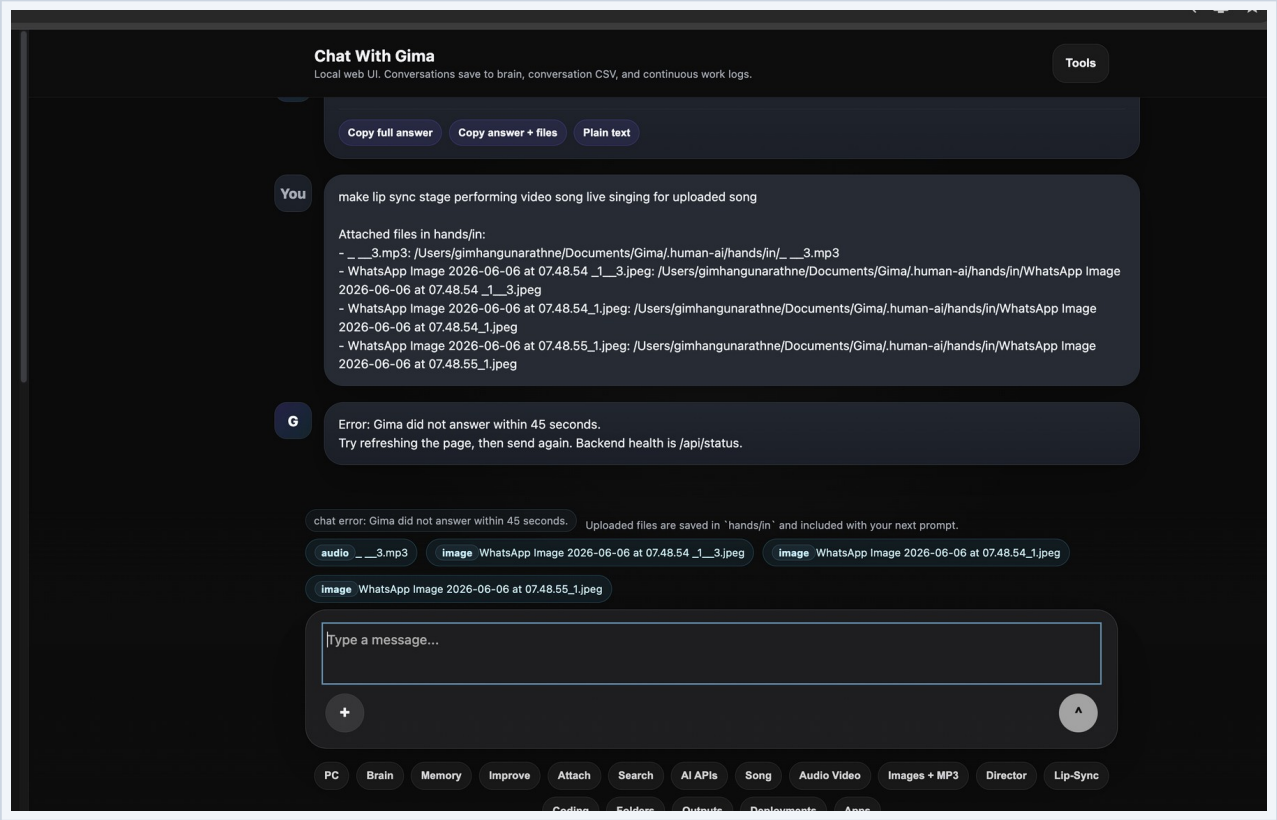
Screenshot 1 - Artifact/report workflow: Gima identifies a table/report request and generates local CSV/PDF style outputs.

Product Screenshots



Screenshot 2 - Chat workspace and tools: copy/export controls, tool shortcuts, and the provider-aware workspace layout.

Product Screenshots



Screenshot 3 - Media workflow controls: uploaded audio/image inputs and dedicated Audio Video, Director, Images + MP3, and Lip-Sync routes.

Model, Browsing, and Artifact Strategy

Gima uses a hybrid model approach. The local model handles private fallback and lightweight chat. Cloud providers are optional and used when stronger reasoning or model diversity is needed.

Capability	Current Direction	Reliability Rule
OpenRouter and MiniMax	OpenAI-compatible model gateway using a local OPENROUTER_API_KEY placeholder	Never publish real keys; store secrets locally.
Browsing	Search, URL import, and current weather routes	Current facts should use current sources, not stale memory.
Artifacts	CSV, Markdown, PDF, JPG, and spreadsheet-style outputs	Generated files should be saved in visible output folders.
Media	Song, image-plus-audio, director, and lip-sync plans	Use local preview outputs first; stronger video APIs can be connected later.

Recommended next improvements: streaming responses, usage/cost logging, provider benchmark table, and a model picker in settings.

Media Workflows

Gima is moving toward an advanced AI video-song workflow. The practical path is staged: analyze the prompt and audio, create a director plan, generate scene and camera metadata, then call a suitable video/lip-sync backend when available.

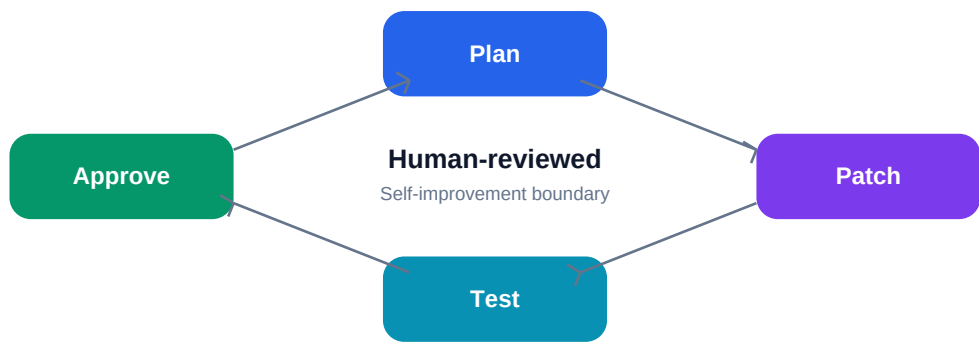
Stage	Function	Output
Director plan	Convert prompt into scenes, camera angles, emotion, lighting, and performance notes	scene_plan.md, shot_list.csv
Audio analysis	Estimate duration, beats, sections, pitch mood, and vocal timing	audio_analysis.json
Video draft	Generate storyboard, still frames, or image-to-video calls	preview assets and manifest
Lip-sync	Use selected face image and audio with a dedicated lip-sync model or API	rendered video path plus logs

Gima should present these as visible stages so the user can see progress instead of waiting on a black-box 45-second timeout.

Safety, Governance, and Improvement

Gima should learn continuously from user-approved memory and public sources, but code upgrades must stay review-gated. A safe improvement system separates learning from self-modifying code.

Figure 2 - Review-Gated Improvement Loop



Risk	Control
Secret leakage	Never commit API keys; use local secrets files and masked UI labels.
Bad automation	Use approval gates for code, GitHub sync, money-related actions, and public posting.
Stale facts	Route current queries through browsing or APIs.
Runaway updates	Use schedules, logs, tests, and human review instead of uncontrolled self-updates.

Roadmap

Phase	Milestone	Result
1	Stabilize chat, browsing, OpenRouter, weather, and artifact generation	Reliable AI workspace basics
2	Add screenshot-rich exports, Excel workbooks, source registers, and costing templates	Professional business outputs
3	Build staged media director pipeline with audio analysis and video backend adapters	Practical music/video workflow
4	Add agent builder, bot templates, scheduler, and GitHub sync dashboards	Repeatable automation
5	Add public portfolio mode, LinkedIn content assistant, and legal earning workflows	Commercial-ready presence

LinkedIn summary

I am building Gima, a local-first AI workspace that combines memory, browsing, artifact generation, model routing, and review-gated self-improvement. The project focuses on making AI inspectable: users can see what sources were used, where outputs were saved, and which model or workflow handled the task.

Conclusion

Gima is strongest when it is treated as an operating workspace for AI work: memory, files, tools, models, media, and governance in one visible system. The next step is to keep converting fragile prompts into reliable routes with clear outputs.